CHAPTER 1

Intro — The Fundamentals

Data Representation, Architecture, Operating Systems & Software Translation

LEARNING OBJECTIVES

- Binary magnitudes, binary prefixes (kibi, mebi), decimal prefixes
- Binary addition and subtraction (sign & magnitude, complements)
- Image resolution, color depth, and vector/bitmap formats
- File compression methods (lossy vs. lossless)
- Fetch-Execute cycle and roles of PC, MDR, MAR, ACC, CIR
- Language translators: Assemblers, Compilers, Interpreters
- Operating System roles & Key Management Tasks
- Interrupts, Peripheral connections (USB, HDMI, VGA), IDE features
- Binary, denary, and hexadecimal number systems
- Hexadecimal and Binary Coded Decimal (BCD) uses
- Representation of sound, sampling rate, and resolution
- The basic Von Neumann model of a computer system
- High-level vs. Low-level languages, source vs. machine code
- Stages of a two-pass assembler & assembly code tracing
- Utility software (deframag, virus checker, backup, formatting)

KEY TERMS

Binary: Base two number system based on the values 0 and 1 only.

Bit: Abbreviation for binary digit.

One's complement: Each binary digit in a number is reversed to allow positive & negative representation.

Two's complement: Each bit is reversed and 1 is added in right-most position for negative numbers.

Sign and magnitude: Left-most bit represents the sign (0=pos, 1=neg); remaining bits store value.

Hexadecimal: Base-16 system using denary digits 0–9 and letters A to F.

Memory dump: Contents of computer memory output to screen or printer.

Binary-coded decimal (BCD): System using 4 bits to represent each denary digit.

ASCII code: Coding system for keyboard characters and control codes.

Character set: Defined list of characters recognized by hardware and software.

Unicode: Universal character code representing global languages (first 128 match ASCII).

1.1 Number Systems & Data Representation

Computers consist of millions of tiny transistors acting as electronic switches that are either **ON (1)** or **OFF (0)**. Because of this, computers naturally speak **Binary** (Base-2).

Prefixes & Magnitudes

We measure digital storage using two standards: *Decimal (SI) prefixes* based on powers of 10, and *Binary (IEC) prefixes* based on powers of 2.

Prefix Type	Name	Symbol	Value	Exact Bytes
Decimal (SI)	Kilobyte / Megabyte	KB / MB	10^3 / 10^6	1,000 / 1,000,000
Binary (IEC)	Kibibyte / Mebibyte	KiB / MiB	2^{10} / 2^{20}	1,024 / 1,048,576
Binary (IEC)	Gibibyte / Tebibyte	GiB / TiB	2^{30} / 2^{40}	1,073,741,824 / 1.099×10^{12}

Signed Binary Representation

To represent negative numbers, we use three distinct techniques:

1. **Sign & Magnitude:** Most Significant Bit (MSB) denotes sign (0 = +, 1 = -). E.g., 10000101 = -5.
2. **One's Complement:** Flip all bits of the positive number (05 = 00000101 → 11111010 = -5).
3. **Two's Complement:** Flip all bits and add 1 to the right-most bit. E.g., +5 (00000101) → Flip (11111010) → Add 1 = 11111011 (-5). Range for 8 bits: **-128 to +127**.

Hexadecimal & Binary Coded Decimal (BCD)

Hexadecimal (Base-16) makes long binary strings human-friendly (e.g., HTML color #FF0000, MAC addresses, and **Memory Dumps** during error debugging). Each Hex digit represents exactly 4 bits (a nibble).

Binary Coded Decimal (BCD) encodes each individual denary digit (0–9) using 4 bits. For instance, denary 798 becomes 0111 1001 1000. BCD is widely used in financial hardware (calculators, digital clocks) to avoid rounding errors.

1.2 Multimedia Representation & Compression

Bitmap Images

Images are stored as a grid of pixels. **Image Resolution** is the number of pixels per unit length (or width × height dimensions). **Color Depth** is the number of bits used per pixel.

FIGURE 1.2 — COLOR DEPTH VS BIT REQUIREMENT

Colors Supported	Bits per Pixel (bpp)	Calculation Formula
2 colors (Monochrome)	1 bit	$2^1 = 2$
16 colors	4 bits	$2^4 = 16$
256 colors	8 bits (1 byte)	$2^8 = 256$
16.7 Million (True Color)	24 bits (3 bytes)	$2^{24} \approx 16.7M$

File Size Formula: `File Size (bits) = Width × Height × Color Depth (bits)`.

Sound Representation

Sound is analogue continuous audio waves. Computers convert it into digital data via an ADC (Analogue-to-Digital Converter) using two key settings:

- **Sampling Rate:** Number of sound samples taken per second (Hz). Higher rate = closer match to original sound wave.
- **Sampling Resolution:** Number of bits allocated per sample. Higher resolution = greater dynamic range and accuracy, but larger file size.

Compression Methods

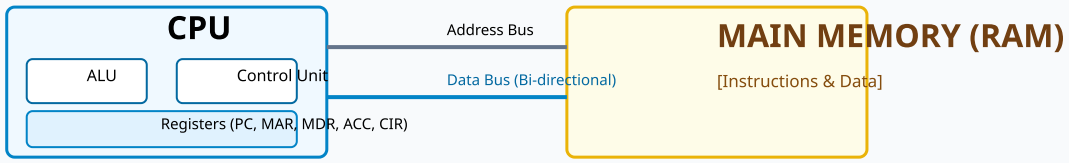
Lossy Compression: Removes unnoticeable data permanently (e.g., MP3 audio, JPEG images). Reduces file size drastically but loses original detail.

Lossless Compression: Compresses file without losing any data (e.g., Run-Length Encoding RLE, ZIP files). Reversible, essential for text, executable code, and raw database files.

1.3 Computer Architecture & The Von Neumann Model

The **Von Neumann Architecture** introduces the concept of the *Stored Program Computer*, where both data and instructions share the same memory space and system bus system.

FIGURE 1.3 — VON NEUMANN COMPUTER ARCHITECTURE MODEL



KEY CPU REGISTERS

- **PC (Program Counter):** Holds address of the *next* instruction to be fetched.
- **MAR (Memory Address Register):** Holds memory location currently being read/written.
- **MDR (Memory Data Register):** Holds data read from or written to memory.
- **ACC (Accumulator):** Stores results of arithmetic and logic calculations from ALU.
- **CIR (Current Instruction Register):** Stores instruction currently being decoded/executed.

The Fetch-Execute (F-E) Cycle

The fundamental operational loop of the CPU works in six distinct steps:

FIGURE 1.4 — SEQUENTIAL ORDER OF THE F-E CYCLE

1. The address contained in the **PC** is copied to the **MAR**.
2. The instruction is copied from the memory location in **MAR** and placed in **MDR**.
3. The value in the **PC** is incremented by 1.
4. The instruction is copied from the **MDR** and placed in the **CIR**.
5. The instruction is decoded.
6. The instruction is executed.

Interrupts & Peripheral Connections

An **Interrupt** is a signal sent from a device/software requesting CPU attention. When received, the CPU suspends current execution to service the interrupt. Computers connect to external devices using standardized ports:

- **USB (Universal Serial Bus):** High-speed serial connection supporting plug-and-play peripherals.
- **HDMI / VGA:** Video interface standards (HDMI transmits digital video+audio, VGA transmits legacy analogue video).

1.4 Operating Systems & Software Architecture

An **Operating System (OS)** provides a software platform hiding hardware complexity. Its core management tasks include:

OS Task	Primary Responsibilities
Memory Management	Allocates RAM space to active processes, handles virtual memory paging.
Process Management	Schedules CPU time among running threads, avoids deadlock.
Hardware/Device Management	Communicates with hardware using low-level device drivers.
Security Management	Manages user authentication, file access permissions, and passwords.

Utility Software

Utilities perform specific maintenance tasks to keep a system optimized:

Disk Formatter: Prepares storage media for first use by wiping and initializing file systems. **Virus Checker:** Scans storage for malware signatures. **Defragmentation Software:** Rearranges fragmented file clusters on HDDs sequentially for faster read speed. **Backup Utilities:** Automatically copies critical system files (e.g., File History, Time Machine).

1.5 Language Translators & Code Execution

Computers only execute **Machine Code** (binary). Software written in High-Level Languages (Python, Java) or Low-Level Assembly requires translation.

Translator	Input Source Code	Translation Method	Benefits / Drawbacks
Compiler	High-Level Language	Translates entire program at once into standalone machine code executable.	+ Fast execution speed. - Debugging takes longer.
Interpreter	High-Level Language	Translates & executes source code line-by-line.	+ Easy, instant debugging. - Slower overall execution.
Assembler	Assembly Code	Converts low-level mnemonics directly into 1-to-1 binary machine code.	+ Direct hardware control. - Hardware-dependent.

Two-Pass Assemblers & IDE Features

A **Two-Pass Assembler** scans code twice: *Pass 1* builds a Symbol Table mapping labels to memory addresses; *Pass 2* converts mnemonics into opcode/operand binary values using the table.

An **IDE (Integrated Development Environment)** speeds up coding with tools: Auto-completion, Syntax Highlighting, Integrated Debuggers (breakpoints/single-stepping), and Built-in Translators.

End of Chapter Exam-Style Questions

1. **a)** The following bytes represent binary integers using the two's complement form. State the equivalent denary values:

i) 0 1 0 0 1 1 1 1 **[1 mark]**

ii) 1 0 0 1 1 0 1 0 **[1 mark]**

iii) Write the integer **−53** in two's complement form. **[2 marks]**

iv) Write the maximum possible range of numbers using the two's complement form of an 8-bit binary number. Give your answers in denary. **[2 marks]**

b) i) Write the denary integer **798** in binary-coded decimal (BCD) format. **[1 mark]**

ii) Write the denary number that is represented by the following BCD number: 1001 0111 0111 0110

c) Give one use of binary-coded decimal system. **[1 mark]** **[2 marks]**

2. A software developer is using a microphone and a sound editing app to collect and edit sounds for his new game. When collecting sounds, the software developer can decide on the sampling resolution he wishes to use.

a) i) State what is meant by *sampling resolution*. **[1 mark]**

ii) Describe how sampling resolution will affect how accurate the stored digitised sound will be. **[2 marks]**

b) The software developer will include images in his new game.

i) Explain the term *image resolution*. **[1 mark]**

ii) The software developer is using 16-colour bit-map images. State the number of bits required to encode data for one pixel of his image. **[1 mark]**

iii) One of the images is 16384 pixels wide and 512 pixels high. The developer decides to save it as a 256-colour bit-map image. Calculate the size of the image file in **gibibytes**. **[3 marks]**

3. Write these six stages of the Von Neumann fetch-execute cycle in the correct order: **[6 marks]**

- *– instruction is copied from the MDR and is placed in the CIR*
- *– the instruction is executed*
- *– the instruction is decoded*
- *– the address contained in PC is copied to the MAR*
- *– the value in PC is incremented by 1*
- *– instruction is copied from memory location in MAR and placed in MDR*

4. File History and Time Machine are examples of back-up utilities offered as part of two different operating systems. Explain why it is important to back up files on a computer. **[3 marks]**

5. Assemblers, compilers and interpreters are all used to translate programs. Discuss the different roles played by each translator. **[6 marks]**